Kavach v1: Moyo implementation profile

How Moyo's Expo app and Cloudflare Worker implement the Kavach v1 protocol: platform adapters, server mapping, retention values, product behaviour and release gates.

Companion to "Kavach v1: protocol specification", revision 4 (formerly SealedStream SS5). October 2026. Nothing in this document changes a wire byte.

How to read this
Section 1 lists the choices this profile makes where the protocol leaves a decision to the host. Section 2 gives the retention policy. Section 3 is the client side: Expo adapters for secrets, the store, recovery and background work. Section 4 is the server side: the Worker, D1 and the Durable Objects. Section 5 is product behaviour the protocol depends on. Section 6 is the release gates and device tests, and section 7 points at the build sequence.
Read the protocol specification first. This document assumes its terms and section numbers.

Contents

1. What this profile decides
2. Retention policy
3. Client adapters (Expo)
4. Server (Cloudflare Worker)
5. Product behaviour the protocol depends on
6. Release gates and device tests
7. Build order

1. What this profile decides

The protocol specification states requirements and leaves the host to choose how to meet them. Moyo's choices are:

Decision left to the host	Moyo's choice
Protected secret store	Expo SecureStore with device-only accessibility for `accountSeed`, `deviceSeed`, `recoveryKey`, `storageKey` and the store instance id.
Encrypted transactional store	Expo SQLite with SQLCipher, keyed by `storageKey`, WAL, `synchronous = FULL`. Write transactions use `BEGIN IMMEDIATE`; the outbound handoff is initiated inside the transaction.
Cross-process writer lock	SQLite's own file lock under `BEGIN IMMEDIATE`. No native lock module.
Platform recovery provider	Apple synced Keychain on iOS. Android Block Store with the end-to-end encryption check on Android, required for release.
Directory and relay	A Cloudflare Worker: D1 for the directory, one `UserDO` per author as the relay, `CircleDO` for presence and wake-ups.
Notifications	Expo push. Visible for a join request (to the owner) and a check-in (to each recipient); silent otherwise.
Reset authority	An operational key as a Worker secret; a management key kept offline that signs a new key list every 14 days with a 30-day lifetime.
Retention	The values in section 2, supplied to `open()` as a versioned policy.
Sync interval	60 seconds while the app is online and running, plus hints and background wake-ups.

Everything the protocol forbids remains forbidden here. Where this document and the protocol disagree, the protocol wins and this document has a bug.

2. Retention policy

Moyo passes the following `RetentionPolicy` to `open()`. The values are proposals to confirm before release; the shape is fixed by the protocol.

Field	Proposed value	Meaning
`deliveryWindowMs`	24 hours	How long the relay keeps a record for readers that have not acknowledged it, from acceptance. One window for locations and check-ins keeps expiry a prefix per stream. A reader away longer returns to an empty trail until the next update, which arrives within a minute from an online author.
`appendRetryWindowMs`	7 days	How long the relay keeps the last-append outcome per stream, so a phone that was offline can finish an uncertain upload with the original result.
`locationHistoryMs`	24 hours	How long a phone keeps location history, counted from `sentAt`.
`checkInHistoryMs`	30 days	How long a phone keeps check-in history.

Field	Proposed value	Meaning
`hygieneRotationMs`	30 days	Rotate an epoch that nothing else has rotated. Daily was rejected: a re-wrap to every recipient every day for no realistic threat.
`maxOutboxRecords`	2,000	Backpressure on the outbox.
`maxOutboxBytes`	4 MiB	Backpressure on the outbox.

Delivery deadlines use the relay's acceptance time plus the window; the Worker enforces it. History retention counts from `sentAt`; the encrypted `capturedAt` is what the user sees. The live pin is the newest kind 0 record in history, so a pin older than `locationHistoryMs` leaves the map and the person shows as last seen with no position. Acknowledgements are coalesced: at most one `ackThrough` per stream per fetch batch, delayed at most 60 seconds, which is a Moyo setting and not a policy field. Changing the policy purges only data outside the new windows, with an explicit disposition for undelivered records. Past days in R2 and the server-side trail in the user's Durable Object disappear from Moyo's backend; the trail a reader sees is the one its own phone received.

3. Client adapters (Expo)

Secrets

`accountSeed`, `deviceSeed`, `recoveryKey`, `storageKey` and the 16-byte store instance id live in Expo SecureStore with device-only accessibility. Only the `recoveryKey` is ever placed in a synced platform item, through the recovery provider below. The database file, device seed, counters and outbox are excluded from iCloud backup and Android Auto Backup.

Store

One SQLite database opened through Expo SQLite with SQLCipher, keyed by the hex `storageKey`, in WAL mode with `synchronous = FULL`. SQLCipher gives encryption at rest for every row, so there is no second per-value encryption layer. The `meta` row binds schema version 1, the store instance id, `identityId` and the current `certificateHash`, as the protocol requires.

Every SDK write runs inside `BEGIN IMMEDIATE`. SQLite's file lock then serialises writers across the foreground app, the background location task and any headless context, without a native module. The adapter's `transaction(fn)` lets the callback start the transport's `send` before the transaction commits, which is what the protocol's handoff rule (T6h) needs; the callback never awaits the response inside. The busy timeout is finite, and a writer that cannot acquire the lock returns `RETRYABLE`.

Recovery providers

iOS. A native Keychain generic-password item with `kSecAttrSynchronizable` true, `kSecAttrAccessibleWhenUnlocked` and the app's access group, keyed by account name plus identity id, holding the 32-byte `recoveryKey`, through a small config plugin. Ordinary Expo SecureStore does not establish iCloud sync. Logout erases local access without deleting the shared item; `eraseLocalAccess()` never means delete the cloud entry. Reset saves a new entry under the new identity id.

Android. A native Block Store provider, required before the Android release. The entry is named `"ss5.recovery." || hex(identityId)` with value `u8(1) || B(accountName) || identityId[32] || recoveryKey[32]`. Before enabling cloud backup, await `isEndToEndEncryptionAvailable()`; only `true` permits `storeBytes` with cloud backup. On false, error or absent services the existing entry is left untouched and a visible reason is returned; never retry by

writing the same entry without cloud backup, which can delete a prior cloud copy. A successful store reports `backupState = pending`. The provider never advertises Keystore or SecureStore as synced recovery and never places the secret in unprotected Auto Backup.

Clock, background work and transport

A clock adapter keeps a durable time high-water mark and monotonic elapsed time for the reset-key list rules. The background location task calls `enqueueLocation` and then `sync`. The transport is Moyo's authenticated fetch and sockets; its `send` queues the bytes synchronously and returns a promise, so it can be initiated inside T6h. The app's `observe` handler forwards online, offline, resume and the Worker's metadata hints.

4. Server (Cloudflare Worker)

Component	Responsibility
Directory (D1)	Accounts keyed by `accountName` with the current certificate pointer and generation; immutable certificates by hash; snapshots by `circleId` with compare-and-swap on revision and hash; vaults by account with compare-and-swap on revision and hash; pending join and leave requests by circle and account, expiring after 7 days; reset receipts by `resetId`; the current reset-key list. Sessions authenticate the account and its current device; a superseded device is refused every write. Activation emits a metadata hint to every circle the account is in.
Relay (`UserDO`)	One Durable Object per author: current epoch and fence; the author's retained records keyed by `deliverySeq` with their acceptance time; one subscription row per reader device with its `ackCursor`; the `expiredThrough` watermark; the manifests and wraps of every epoch that is current or still referenced by a retained record; the per-stream high-water (epoch, counter) pair and last-append outcome. `readRecords` filters by the subscription's certificate. `appendRecord`, `readRecords` and `ackThrough` run here. Deletion on a cursor advance runs inline in the same transaction; the expiry prefix advances on an alarm. The Durable Object's single-threaded storage transaction gives the serialisation the protocol requires without extra locking.
Fan-out (`CircleDO`)	Presence and wake-ups only: on append, notify each recipient's socket that its mailbox has new sequence numbers. It stores no record.
Notifications	Silent pushes wake a closed app for locations and key handoffs. Two events are visible notifications through Expo push: a join request, sent to the owner with the requester's name, and a check-in, sent to each recipient with the sender's name and the app's generic wording. Neither carries a coordinate or a payload. Notification permission is asked for these two and the app's own alerts.
Reset authority	An endpoint that, after fresh re-authentication, signs the type 7 receipt with the active operational key held as a Worker secret. The management key signs a new type 8 list offline every 14 days with a 30-day lifetime; `app.config.ts` pins the management public key and the minimum list version. A standby operational key stays offline until the active one is retired by a new list.
Removed	The server-side trail in the user's Durable Object, past days in R2, device lists, pending devices, key requests, archives and the circle log tables. The Worker keeps no location after delivery.

`./scripts/seed.sh` creates two accounts and a circle against the local Worker. The reference host in the SDK repository's `examples/host-worker` is the pattern for these endpoints and is tested in workerd.

5. Product behaviour the protocol depends on

- **Signup never asks for the recovery code.** The account is created, the provider saves the key, and the Account screen shows a recovery status from `backupState`: backed up to iCloud, backed up to Google, or not backed up yet. Only in the last case does the app later prompt, gently, to save the code. The code is always available in settings behind local authentication.
- **A new phone shows people as they arrive.** After `restoreAccount`, the map shows each circle member as "waiting for their phone" while that author's current epoch has no key for the new device, until the author's next sync delivers one. Online authors arrive within a minute; offline ones when they wake. The app never shows a blank map or a stale pin as live.
- **A pin older than a day leaves the map.** With a 24-hour history and a 24-hour delivery window, a person silent for a day shows as last seen with no position rather than a day-old dot. A reader back after a weekend sees each person again on their next update.
- **Join requests notify the owner.** The owner's approval screen opens from the notification. Approval is one tap; the requester starts sharing on approval unless they turned that off when requesting.
- **Check-ins notify recipients visibly.** The notification carries the sender's name and generic wording; the app decrypts the check-in when opened. Locations never notify.
- **Leaving a circle takes effect locally at once.** The leaver stops sending and stops seeing that circle's members immediately; the owner's removal follows when the owner is next online.
- **One active phone.** Setting up the app on a second phone retires the first. The settings screen says so before activation.
- **Reset is a last resort with a warning.** The reset screen states that circles owned by this account will close and that every other circle will need to re-admit the account.

6. Release gates and device tests

These tests are in addition to the protocol's conformance package and scenario list. Each is run on real devices and its result recorded with device, OS and Play services versions.

Gate	Evidence required
Android recovery	Restore the Play-distributed app and its Block Store entry to a second supported Android device through cloud restore, with no old phone and no code. Decrypt the vault, preserve owner and member identity, activate fresh device keys, receive future updates. Also exercise end-to-end check false and error, absent Play services, no screen lock, backup disabled or delayed, wrong Google account, wrong Moyo account, missing entry, capacity exhaustion, process death during save, and retirement without deleting the shared entry.
iOS recovery	The same restore through synced Keychain to a second iPhone, with iCloud Keychain off and delayed, wrong Apple account, app reinstall and biometric changes.
Durable state	The protocol's crash outcomes against the real SQLCipher database on both platforms. Race the foreground app and the background location task through `BEGIN IMMEDIATE`. The handoff race with the real transport adapter: an authorization change committed by one context while the other is between its check and its send never produces an upload sealed for the old audience.

Gate	Evidence required
Lost phone	With the complete old-phone state in the attacker's hands, after activation and every author's rotation the attacker opens nothing new.
Relay	In workerd: a record is deleted when the last required reader's cursor passes it and at the window without one; an equal or older cursor writes nothing; expiry advances as a prefix; a retried append inside the window returns the original outcome after deletion; a stale append is refused without reinsertion; a cursor above the tail is rejected; a replacement device's subscription excludes records sealed to the old certificate; members get subscriptions when an author opens a new stream; epoch blobs go only when no record references them and the epoch is not current; a check-in survives a later location; a join request and a check-in each produce exactly one visible notification with no coordinate. Measured: Durable Object writes per delivered record at 1, 10 and 100 readers, recorded before and after acknowledgement coalescing.
Performance	On one mid-range iOS and one mid-range Android device: full 100-account rotation, 100-person receiving load, cold start, activation and time to a full map with five authors online, storage growth, memory and battery, p50 and p95 recorded. No unmeasured speed claim appears in release notes.
End to end	Maestro flows for create, join with the owner's notification, approve, phone replacement on a second emulator with per-person waiting pins that fill in, removal and a check-in notification.

An independent review of the composed protocol, the recovery flows, the store binding, the handoff serialisation and the Worker's atomicity precedes production.

7. Build order

The build sequence document lists nineteen steps from primitives to Moyo integration, each with what to write, what to test and what comes next. The order is: primitives and codec against the vectors; identity, vault, membership and audience; epochs, records, the store and the two sides of delivery; reset; the client; the Expo adapters; the reference host; the fault harness; benchmarks; and finally the Moyo integration, which replaces `packages/sealed`, reshapes `UserDO` and `CircleDO`, removes the server-side history, and adds the Account settings entries for recovery status and the recovery code.